

DESARROLLAR SOFTWARE A PARTIR DE LA INTEGRACIÓN DE SUS MÓDULOS COMPONENTES

GA8-220501096-AA1-EV01

PROYECTO BIDA – ODONTOLOGÍA ESPECIALIZADA

Evidencia de desempeño

Aprendiz: Carlos Andrés Cajas Cerón

Ficha: 3118326

Programa de formación: Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje – SENA

1. INTRODUCCIÓN

El presente documento corresponde a la evidencia de desempeño GA8-220501096-AA1-EV01, denominada “Desarrollar software a partir de la integración de sus módulos componentes”.

La evidencia presenta el proceso de desarrollo e integración del proyecto BIDA – Odontología Especializada, una aplicación web orientada a la gestión de los diferentes procesos relacionados con una clínica odontológica.

El proyecto está compuesto por diferentes módulos funcionales que permiten gestionar información de pacientes, odontólogos, empleados, tratamientos, citas, facturación, consultorios, equipos, locales y detalles de tratamientos, entre otros componentes.

Para el desarrollo de la aplicación se utilizaron tecnologías como HTML, CSS, JavaScript, Node.js, Express y MySQL. La comunicación entre la interfaz de usuario y la base de datos se realiza mediante servicios web desarrollados con una API REST, permitiendo ejecutar operaciones de consulta, creación, actualización y eliminación de información.

La integración de los componentes permite que el usuario pueda iniciar sesión, acceder al panel principal y posteriormente utilizar los diferentes módulos del sistema. De esta manera, los componentes desarrollados funcionan como parte de una aplicación web integrada y no como elementos independientes.

Durante el desarrollo también se realizaron pruebas unitarias y pruebas de funcionamiento de los módulos, utilizando herramientas como Jest, Thunder Client y el navegador web, con el propósito de verificar el correcto funcionamiento de las rutas, servicios y componentes desarrollados.

Finalmente, se documentan las herramientas utilizadas, la estructura del proyecto, la integración entre sus componentes, la configuración del ambiente de desarrollo y las evidencias que permiten demostrar el funcionamiento de la aplicación.

2. OBJETIVOS

2.1 Objetivo general

Desarrollar e integrar los módulos componentes del sistema BIDA – Odontología Especializada, utilizando tecnologías web, servicios API y una base de datos relacional, con el propósito de obtener una aplicación funcional que permita gestionar de manera organizada la información y los procesos de una clínica odontológica.

2.2 Objetivos específicos

- Integrar los diferentes módulos funcionales que conforman el proyecto BIDA – Odontología Especializada.
- Implementar la comunicación entre la interfaz web, los servicios API y la base de datos MySQL.
- Utilizar JavaScript para desarrollar la interacción y funcionalidad de los diferentes componentes de la aplicación.
- Implementar servicios web mediante Node.js y Express para gestionar las operaciones realizadas sobre la información almacenada.

- Integrar un sistema de inicio de sesión que permita validar la información del empleado y acceder al panel principal de la aplicación.
- Implementar operaciones CRUD para la gestión de la información de los diferentes módulos del sistema.
- Realizar pruebas unitarias y pruebas de funcionamiento para verificar el comportamiento de los módulos desarrollados.
- Utilizar Git como herramienta de control de versiones para gestionar el código fuente del proyecto.
- Documentar el ambiente de desarrollo, las tecnologías utilizadas, la estructura del proyecto y el proceso de integración de sus componentes.

3. DESCRIPCIÓN DEL PROYECTO BIDA – ODONTOLOGÍA ESPECIALIZADA

BIDA – Odontología Especializada es una aplicación web desarrollada como proyecto formativo para apoyar la gestión de los procesos administrativos y operativos relacionados con una clínica odontológica.

La aplicación permite organizar y consultar información correspondiente a diferentes entidades del sistema mediante módulos independientes que se encuentran integrados dentro de una misma plataforma.

Entre los principales módulos desarrollados se encuentran:

- Pacientes.
- Odontólogos.
- Empleados.
- Citas.
- Tratamientos.
- Detalle de tratamientos.
- Facturación.
- Consultorios.
- Equipos.
- Locales.

Cada módulo cuenta con componentes de interfaz y servicios que permiten realizar operaciones sobre la información almacenada en la base de datos.

La aplicación utiliza una arquitectura en la cual la interfaz web se comunica con una API desarrollada mediante Node.js y Express. Esta API contiene las rutas correspondientes a cada módulo y permite realizar operaciones mediante métodos HTTP como GET, POST, PUT y DELETE.

La información es almacenada en una base de datos MySQL denominada "bida". La conexión entre el servidor y la base de datos permite que las operaciones realizadas desde la aplicación sean procesadas y respondidas de manera dinámica.

El proyecto también incorpora un sistema de inicio de sesión mediante el cual se valida el documento de identidad y el correo electrónico de un empleado registrado. Una vez realizada correctamente la validación, el usuario es dirigido al panel principal ("index.html"), desde donde puede acceder a los diferentes módulos de la aplicación.

De esta manera, BIDA integra los componentes de frontend, backend, servicios web y base de datos en una aplicación web funcional.

4. REQUERIMIENTOS DEL SISTEMA

Para el desarrollo del proyecto BIDA – Odontología Especializada se tuvieron en cuenta los requerimientos necesarios para gestionar la información relacionada con los procesos principales de una clínica odontológica.

4.1 Requerimientos funcionales

Entre los principales requerimientos funcionales implementados se encuentran:

- Permitir el acceso de empleados mediante un sistema de inicio de sesión.
- Consultar información de los pacientes registrados.
- Registrar nuevos pacientes.
- Actualizar información de pacientes.
- Eliminar registros de pacientes cuando corresponda.
- Gestionar información de odontólogos.
- Gestionar información de empleados.
- Gestionar citas odontológicas.
- Gestionar tratamientos.
- Gestionar información relacionada con la facturación.
- Gestionar consultorios.

- Gestionar equipos.
- Gestionar locales.
- Gestionar detalles de tratamientos.
- Consultar información almacenada en la base de datos.
- Permitir la comunicación entre la interfaz y los servicios API.
- Validar diferentes situaciones de error durante las operaciones realizadas.

4.2 Requerimientos no funcionales

El sistema también considera características relacionadas con su funcionamiento y organización:

- Utilización de una arquitectura organizada por módulos.
- Separación de las rutas API en archivos independientes.
- Utilización de una base de datos relacional.
- Utilización de JavaScript para la interacción de la interfaz.
- Implementación de servicios mediante Node.js y Express.
- Utilización de métodos HTTP para la comunicación con los servicios.
- Organización del código fuente en carpetas y archivos de acuerdo con su función.
- Utilización de Git para el control de versiones.
- Realización de pruebas unitarias sobre los módulos.
- Configuración de un ambiente local para desarrollo y pruebas.

5. HERRAMIENTAS Y TECNOLOGÍAS UTILIZADAS

Para el desarrollo del proyecto BIDA – Odontología Especializada se utilizaron diferentes herramientas y tecnologías orientadas al desarrollo de aplicaciones web, gestión de bases de datos, creación de servicios API, pruebas y control de versiones.

Las principales tecnologías utilizadas fueron:

HTML5: utilizado para la construcción de las interfaces y estructuras de las páginas web del sistema.

CSS3: utilizado para definir los estilos visuales, distribución y presentación de las interfaces.

JavaScript: utilizado para implementar la interacción de las páginas web y la comunicación con los servicios de la aplicación.

Node.js: utilizado como entorno de ejecución para el desarrollo del servidor y los servicios del sistema.

Express: utilizado como framework para la creación y organización de las rutas y servicios de la API.

MySQL: utilizado como sistema de gestión de base de datos relacional para almacenar la información del proyecto.

MySQL2: utilizado para establecer la comunicación entre la aplicación desarrollada con Node.js y la base de datos MySQL.

Visual Studio Code: utilizado como entorno de desarrollo para la creación, organización y edición del código fuente.

Jest: utilizado para realizar pruebas automatizadas de los módulos y verificar el comportamiento de las rutas desarrolladas.

Thunder Client: utilizado para realizar pruebas de los servicios web mediante solicitudes HTTP.

Git y GitHub: utilizados para el control de versiones y almacenamiento del código fuente del proyecto.

Estas herramientas permitieron integrar los diferentes componentes del sistema y facilitaron el desarrollo, ejecución, prueba y mantenimiento del proyecto.

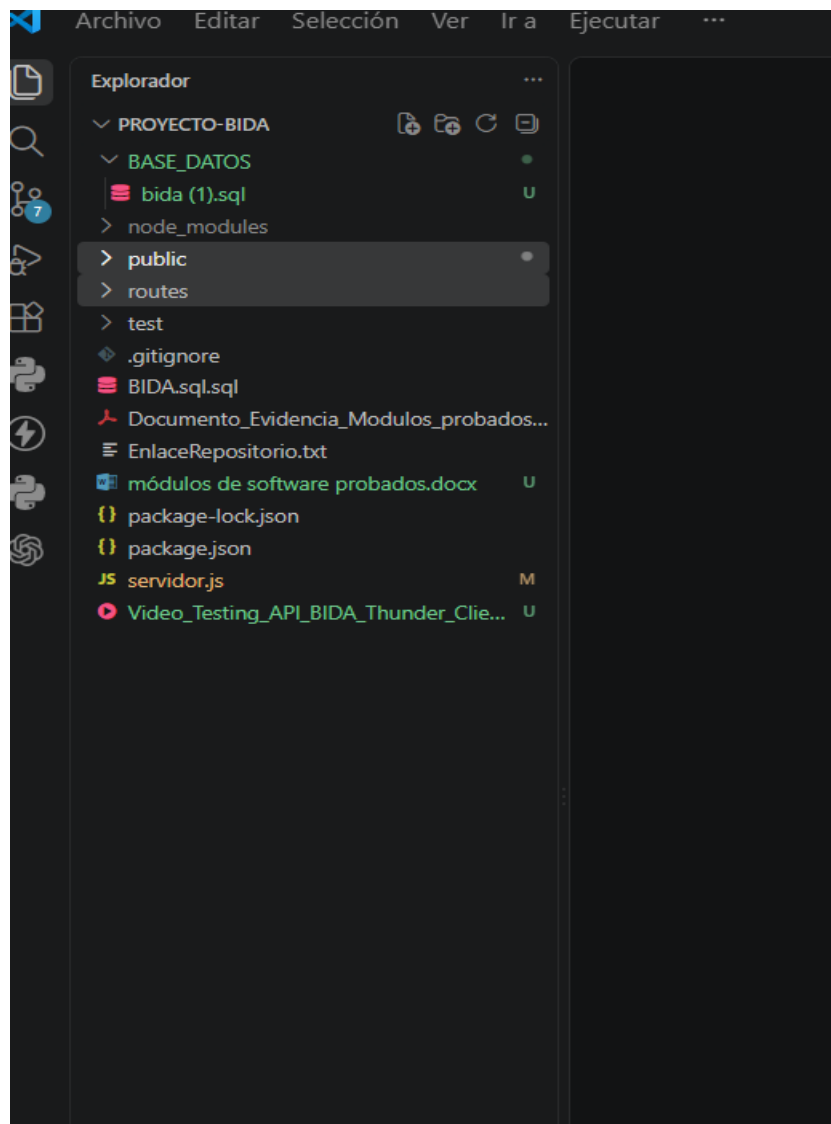


Figura1. Estructura general del proyecto VIDA en visual Studio Code

6. ENTORNO DE DESARROLLO

El proyecto BIDA – Odontología Especializada fue desarrollado y probado en un ambiente local.

El entorno de desarrollo está compuesto por Visual Studio Code como IDE principal, Node.js como entorno de ejecución del servidor, Express para la construcción de los servicios web y MySQL como sistema gestor de base de datos.

La aplicación se ejecuta mediante un servidor local configurado en el puerto 3040. La conexión con la base de datos MySQL se encuentra configurada mediante el puerto 3309.

El ambiente permite ejecutar la aplicación, realizar consultas a la base de datos y probar la comunicación entre la interfaz web, los servicios API y las diferentes tablas de la base de datos.

Durante el desarrollo se utilizaron herramientas adicionales como Jest para las pruebas automatizadas y Thunder Client para comprobar el funcionamiento de los servicios mediante métodos HTTP.



Figura 2. Ejecución del servidor local y conexión con la base de datos

7. ARQUITECTURA E INTEGRACIÓN DEL SISTEMA

El proyecto BIDA – Odontología Especializada se encuentra organizado mediante una arquitectura web en la que intervienen diferentes componentes que trabajan de manera integrada.

La interfaz de usuario corresponde a la capa frontend, conformada principalmente por archivos HTML, CSS y JavaScript ubicados dentro de la carpeta pública del proyecto.

La capa backend está desarrollada mediante Node.js y Express. En esta capa se encuentran las rutas correspondientes a los diferentes módulos del sistema, las cuales permiten recibir solicitudes HTTP y procesar las operaciones requeridas.

La capa de datos está conformada por la base de datos MySQL denominada "bida", donde se almacenan los registros correspondientes a las diferentes entidades del sistema.

El funcionamiento integrado puede representarse de la siguiente manera:

Usuario → Interfaz web → API REST → Node.js/Express → MySQL → Respuesta a la interfaz

Esta integración permite que las acciones realizadas desde la aplicación sean procesadas por los servicios correspondientes y que la información almacenada en la base de datos sea consultada, registrada, actualizada o eliminada de acuerdo con las operaciones disponibles.

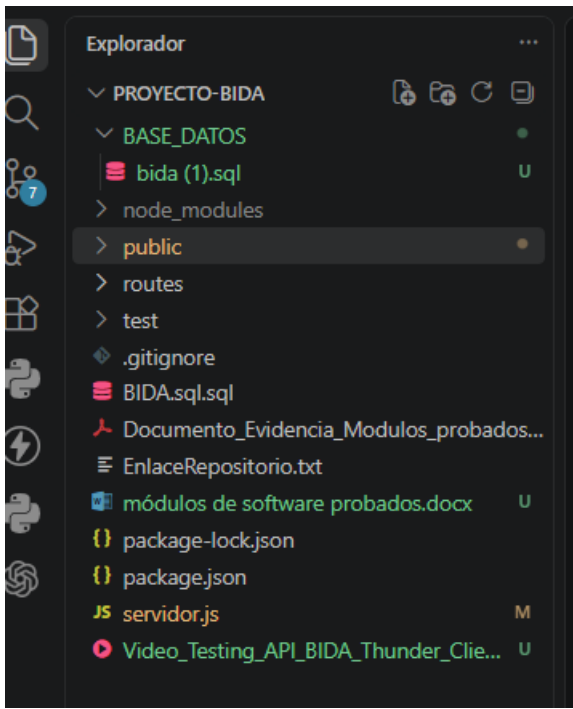


Figura 3. Organización de los componentes frontend y backend del proyecto

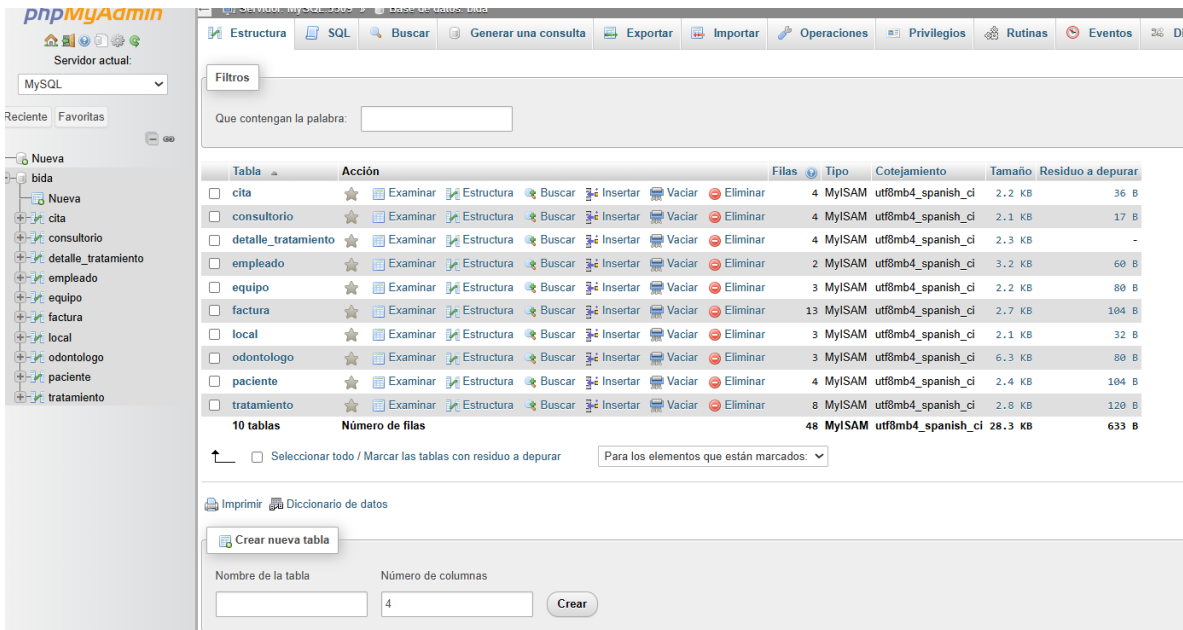


Figura 4. Base de datos BIDA utilizada por el sistema.

8. ESTRUCTURA DE MÓDULOS

El sistema BIDA – Odontología Especializada se encuentra dividido en módulos funcionales que permiten organizar las operaciones relacionadas con la gestión odontológica.

Los principales módulos implementados son:

- Pacientes.
- Odontólogos.
- Empleados.
- Citas.
- Tratamientos.
- Detalle de tratamientos.
- Facturación.
- Consultorios.
- Equipos.
- Locales.

Cada módulo posee rutas específicas dentro del backend y componentes correspondientes en la interfaz web.

La separación de las rutas permite mantener organizado el código fuente y facilita el mantenimiento y las pruebas individuales de cada componente.

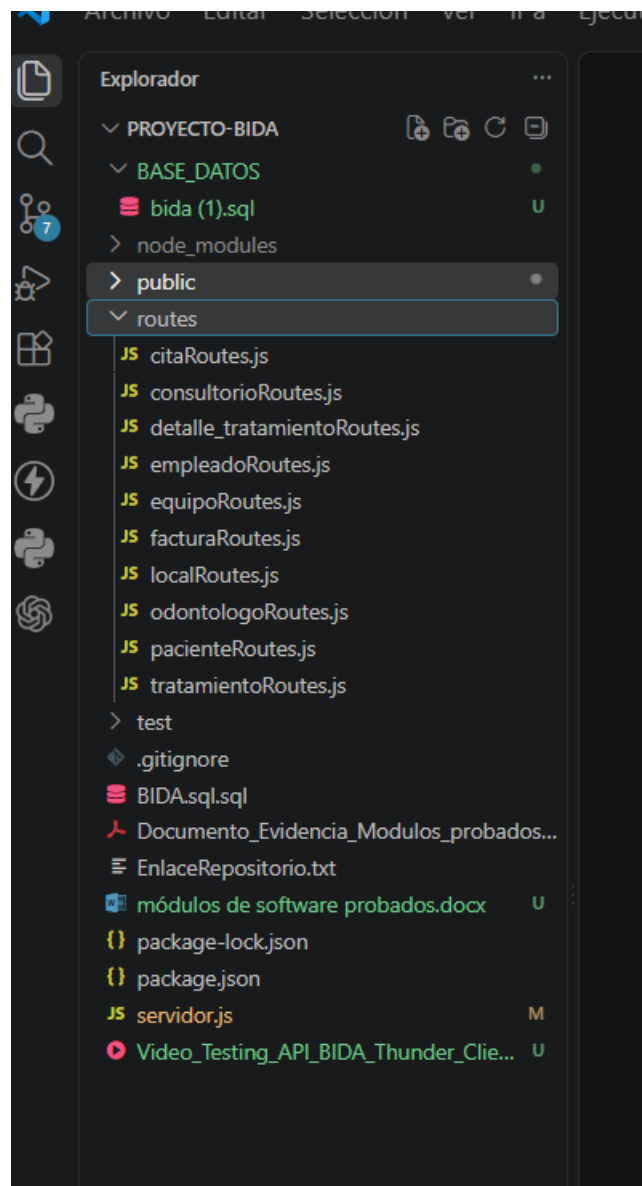


Figura 5. Archivos de rutas correspondientes a los módulos del sistema BIDA

9. INTEGRACIÓN DE LOS MÓDULOS

Los módulos desarrollados para BIDA – Odontología Especializada no funcionan de manera aislada, sino que se encuentran integrados dentro de una misma aplicación web.

El usuario inicia el proceso desde la página de inicio de sesión. Después de validar correctamente el documento de identidad y el correo electrónico registrado para el empleado, el sistema permite el acceso al panel principal.

Desde el panel principal se puede acceder a los diferentes módulos disponibles. Cada módulo se comunica con las rutas correspondientes de la API para consultar o modificar la información almacenada en la base de datos.

La integración permite que los diferentes componentes del sistema trabajen conjuntamente y que la información pueda ser gestionada desde una única plataforma.



The image shows a login form for the BIDA system. The form is centered on a light blue background. It has a white background with rounded corners. At the top, the word "BIDA" is written in large blue letters. Below it, "Iniciar sesión" is written in bold black letters, followed by "Sistema de Odontología Especializada" in a smaller black font. There are two input fields: the first is labeled "Documento de identidad" and contains the number "106178945"; the second is labeled "Correo electrónico" and contains the email "laura@bida.com". Below these fields is a blue button with the text "Ingresar" in white. At the bottom of the form, there is a link that says "¿Olvidaste tus datos?" followed by "Solicitar acceso" in blue text.



Figura 6. Inicio de sesión y acceso al panel principal del sistema.

10. SISTEMA DE INICIO DE SESIÓN

El proyecto cuenta con un sistema de inicio de sesión que permite validar el acceso de los empleados registrados.

El proceso solicita el documento de identidad y el correo electrónico del empleado. Estos datos son enviados mediante JavaScript al servicio correspondiente de la API.

El backend consulta la información del empleado en la base de datos mediante la ruta:

"GET /servidor/empleado/documento/:documento"

Una vez obtenido el empleado, el sistema verifica que el correo electrónico ingresado coincida con el correo registrado.

Cuando la información es correcta, se establece la sesión del usuario mediante "localStorage" y posteriormente se realiza la redirección hacia "index.html".

Este mecanismo permite integrar el componente de autenticación con el panel principal y los diferentes módulos de la aplicación.

```
Explorador
└─ PROYECTO-BIDA
  └─ BASE_DATOS
    └─ bida (1).sql
  └─ node_modules
  └─ public
    └─ # cita.css
    └─ cita.html
    └─ JS cita.js
    └─ # consultorio.css
    └─ consultorio.html
    └─ JS consultorio.js
    └─ detalle_tratamiento.html
    └─ # detalle-tratamiento.css
    └─ JS detalle-tratamiento.js
    └─ # empleado.css
    └─ empleado.html
    └─ JS empleado.js
    └─ # equipo.css
    └─ equipo.html
    └─ JS equipo.js
    └─ # factura.css
    └─ factura.html
    └─ JS factura.js
    └─ # index.css
    └─ index.html
    └─ JS index.js
    └─ # local.css
    └─ local.html
    └─ JS local.js
    └─ # login.css
    └─ login.html M

login.html M
public > login.html > html
2 <html lang="es">
18 <body>
20 <main class="contenedor">
22 <section class="tarjeta">
37 <form id="formLogin" class="formulario">
85 <div>
86 <div>
87 <div>
88 <div>
89 <a href="#">
90 Solicitar acceso
91 </a>
92 </div>
93 </div>
94 </form>
95 </section>
96 </main>
97 </div>
98 </div>
99 </div>
100 </div>
101 <script src="login.js"></script>
102 </body>
103 </html>
104 </html>
105 </html>
```

```
Explorador
└─ PROYECTO-BIDA
  └─ BASE_DATOS
    └─ bida (1).sql
  └─ node_modules
  └─ public
    └─ # cita.css
    └─ cita.html
    └─ JS cita.js
    └─ # consultorio.css
    └─ consultorio.html
    └─ JS consultorio.js
    └─ detalle_tratamiento.html
    └─ # detalle-tratamiento.css
    └─ JS detalle-tratamiento.js
    └─ # empleado.css
    └─ empleado.html
    └─ JS empleado.js
    └─ # equipo.css
    └─ equipo.html
    └─ JS equipo.js
    └─ # factura.css
    └─ factura.html
    └─ JS factura.js

JS login.js M
public > JS login.js > formulario.addEventListener('submit') callback > setTimeout() callback
18 formulario.addEventListener('submit', async (evento) => {
49 }
50
51
52 mensaje.textContent =
53 'Validando información...';
54
55
56 try {
57
58 // =====
59 // BUSCAR EMPLEADO POR DOCUMENTO
60 // =====
61
62 const respuesta = await fetch(
63 'http://localhost:3040/servidor/empleado/documento/${encodeURIComponent(documento)}';
64 );
65
66
67 const datos = await respuesta.json();
68
69
70 // =====
71 // EMPLEADO NO ENCONTRADO
72 // =====
```

```
// =====
// EMPLEADO NO ENCONTRADO
// =====

setTimeout(() => {
  window.location.href =
    'index.html';
}, 1000);

} else {
  mensaje.textContent =
    'El correo no coincide con el empleado.';
```

Figura 7. Código JavaScript utilizado para validar el inicio de sesión y redirigir al panel principal.

11. INTEGRACIÓN CON LA API

La aplicación BIDA – Odontología Especializada utiliza una API REST desarrollada con Node.js y Express para permitir la comunicación entre la interfaz web y la base de datos.

Las rutas de la API se encuentran organizadas en archivos independientes dentro de la carpeta "routes", lo que permite separar las funcionalidades correspondientes a cada módulo.

Cada archivo de rutas recibe la conexión con la base de datos y define los servicios necesarios para gestionar la información del módulo correspondiente.

La API utiliza diferentes métodos HTTP de acuerdo con la operación que se desea realizar:

- GET: permite consultar información.
- POST: permite registrar nueva información.
- PUT: permite actualizar información existente.
- DELETE: permite eliminar registros.

Entre las rutas implementadas se encuentran los servicios correspondientes a pacientes, odontólogos, empleados, facturación, tratamientos, locales, equipos, detalles de tratamientos, consultorios y citas.

Las rutas son incorporadas al servidor principal mediante Express y utilizan el prefijo "/servidor", permitiendo que la interfaz y las herramientas de prueba puedan acceder a los diferentes servicios.

Por ejemplo, para el módulo de empleados se encuentra disponible la ruta:

"GET /servidor/empleado"

También se dispone de una ruta para consultar un empleado mediante su documento:

"GET /servidor/empleado/documento/:documento"

Esta última ruta es utilizada por el sistema de inicio de sesión para consultar los datos del empleado y validar la información ingresada.

La integración de la API permite que las operaciones realizadas desde la interfaz sean procesadas por el servidor y posteriormente ejecutadas sobre la base de datos MySQL.

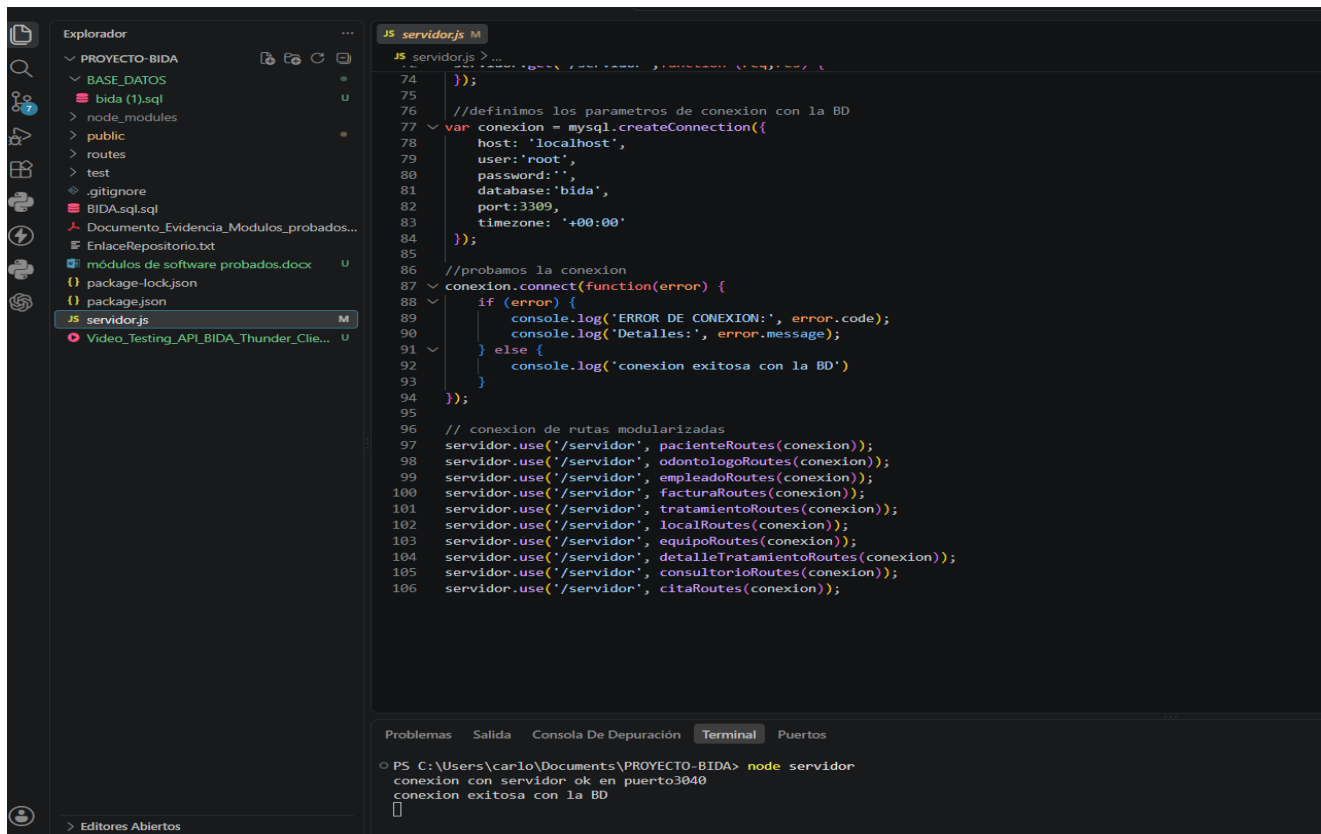
11.1 Organización de las rutas

Las rutas se encuentran separadas por módulos para facilitar la organización, mantenimiento y pruebas del sistema.

Entre los archivos utilizados se encuentran:

- "pacienteRoutes.js"
- "odontologoRoutes.js"
- "empleadoRoutes.js"
- "facturaRoutes.js"
- "tratamientoRoutes.js"
- "localRoutes.js"
- "equipoRoutes.js"
- "detalle_tratamientoRoutes.js"
- "consultorioRoutes.js"
- "citaRoutes.js"

Esta estructura permite que cada módulo tenga sus propios servicios y operaciones, manteniendo una organización modular del backend.



The screenshot shows a code editor with a dark theme. On the left, the 'Explorador' (Explorer) sidebar displays a project structure for 'PROYECTO-BIDA'. The structure includes folders like 'BASE_DATOS', 'node_modules', 'public', 'routes', and 'test'. A file named 'servidor.js' is selected. The main editor area shows the content of 'servidor.js', which includes database connection logic and route registration. The code is as follows:

```
74 });
75
76 //definimos los parametros de conexion con la BD
77 var conexion = mysql.createConnection({
78   host: 'localhost',
79   user: 'root',
80   password: '',
81   database: 'bida',
82   port: 3309,
83   timezone: '+00:00'
84 });
85
86 //probamos la conexion
87 conexion.connect(function(error) {
88   if (error) {
89     console.log('ERROR DE CONEXION:', error.code);
90     console.log('Detalles:', error.message);
91   } else {
92     console.log('conexion exitosa con la BD')
93   }
94 });
95
96 // conexion de rutas modularizadas
97 servidor.use('/servidor', pacienteRoutes(conexion));
98 servidor.use('/servidor', odontologoRoutes(conexion));
99 servidor.use('/servidor', empleadoRoutes(conexion));
100 servidor.use('/servidor', facturaRoutes(conexion));
101 servidor.use('/servidor', tratamientoRoutes(conexion));
102 servidor.use('/servidor', localRoutes(conexion));
103 servidor.use('/servidor', equipoRoutes(conexion));
104 servidor.use('/servidor', detalleTratamientoRoutes(conexion));
105 servidor.use('/servidor', consultorioRoutes(conexion));
106 servidor.use('/servidor', citaRoutes(conexion));
```

At the bottom, the 'Terminal' tab shows the command prompt output:

```
PS C:\Users\carlo\Documents\PROYECTO-BIDA> node servidor
conexion con servidor ok en puerto3940
conexion exitosa con la BD
```


Figura 8. Integración de las rutas de los módulos en el servidor principal.

12. INTEGRACIÓN CON LA BASE DE DATOS

La aplicación BIDA – Odontología Especializada utiliza MySQL como sistema gestor de base de datos.

La base de datos utilizada por el proyecto se denomina "bida" y contiene las tablas necesarias para almacenar la información correspondiente a los diferentes módulos de la aplicación.

La conexión entre Node.js y MySQL se realiza mediante la librería "mysql2". Esta conexión se encuentra configurada en el archivo principal del servidor.

El servidor establece la conexión utilizando los parámetros correspondientes al servidor local, usuario, base de datos y puerto configurado.

Las rutas de la API utilizan esta conexión para ejecutar las operaciones de consulta, inserción, actualización y eliminación de registros.

De esta manera, cuando el usuario realiza una operación desde la interfaz, la solicitud es procesada por la API, la cual ejecuta la consulta correspondiente sobre MySQL y devuelve una respuesta al cliente.

La integración puede representarse de la siguiente manera:

Interfaz web → API REST → Node.js/Express → MySQL → API REST → Interfaz web

Esta integración permite que los módulos trabajen con información almacenada de manera centralizada y que los cambios realizados mediante la aplicación sean reflejados en la base de datos.

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ cita		4	MyISAM	utf8mb4_spanish_ci	2.2 KB	36 B
☐ consultorio		4	MyISAM	utf8mb4_spanish_ci	2.1 KB	17 B
☐ detalle_tratamiento		4	MyISAM	utf8mb4_spanish_ci	2.3 KB	-
☐ empleado		2	MyISAM	utf8mb4_spanish_ci	3.2 KB	60 B
☐ equipo		3	MyISAM	utf8mb4_spanish_ci	2.2 KB	80 B
☐ factura		13	MyISAM	utf8mb4_spanish_ci	2.7 KB	104 B
☐ local		3	MyISAM	utf8mb4_spanish_ci	2.1 KB	32 B
☐ odontologo		3	MyISAM	utf8mb4_spanish_ci	6.3 KB	80 B
☐ paciente		4	MyISAM	utf8mb4_spanish_ci	2.4 KB	104 B
☐ tratamiento		8	MyISAM	utf8mb4_spanish_ci	2.8 KB	120 B
10 tablas	Número de filas	48	MyISAM	utf8mb4_spanish_ci	28.3 KB	633 B

☐ Seleccionar todo / Marcar las tablas con residuo a depurar
 Para los elementos que están marcados:

Nombre de la tabla:
 Número de columnas:

Figura 9. Base de datos VIDA y tablas utilizadas por los módulos del sistema.

13. CONFIGURACIÓN DEL SERVIDOR

El proyecto BIDA cuenta con un servidor local desarrollado utilizando Node.js y Express.

El servidor se encuentra configurado para atender las solicitudes de la aplicación mediante el puerto "3040".

También se encuentra configurada la conexión con la base de datos MySQL mediante el puerto "3309".

El servidor utiliza la carpeta "public" para proporcionar los archivos correspondientes a la interfaz web y permite el acceso a los diferentes servicios de la API mediante el prefijo "/servidor".

La configuración permite ejecutar la aplicación desde un navegador utilizando la dirección local:

"http://localhost:3040"

Al iniciar el servidor se verifica la conexión tanto con el servidor Node.js como con la base de datos MySQL.

Esta configuración permite disponer de un ambiente local para ejecutar, integrar y probar los diferentes componentes de la aplicación.

```
59  });
60  });
61
62  //Asignamos el puerto 3040 a una variable
63  var puerto = 3040;
64
65  //probamos la conexion con servidor local
66  servidor.listen (puerto,function() {
67      console.log('conexion con servidor ok en puerto'+
68          puerto);
69  });
70
71  //crear la primera ruta de acceso con el metodo get
72  servidor.get('/servidor',function (req,res) {
73      res.send('servidor funcionando');
74  });
75
76  //definimos los parametros de conexion con la BD
77  var conexion = mysql.createConnection({
78      host: 'localhost',
79      user:'root',
80      password:'',
81      database:'bida',
82      port:3309,
83      timezone: '+00:00'
84  });
85
86  //probamos la conexion
87  conexion.connect(function(error) {
88      if (error) {
89          console.log('ERROR DE CONEXION:', error.code);
90          console.log('Detalles:', error.message);
91      } else {
```

Problemas Salida Consola De Depuración **Terminal** Puertos

```
PS C:\Users\carlo\Documents\PROYECTO-BIDA> node servidor
conexion con servidor ok en puerto3040
conexion exitosa con la BD
```

Figura 10. Configuración del servidor Node.js y conexión con la base de datos MySQL.

14. ORGANIZACIÓN DEL CÓDIGO

El código fuente del proyecto BIDA se encuentra organizado en diferentes carpetas y archivos de acuerdo con la función que desempeñan dentro de la aplicación.

La carpeta "public" contiene los componentes correspondientes a la interfaz web, incluyendo archivos HTML, hojas de estilos CSS y archivos JavaScript utilizados para la interacción con el usuario.

La carpeta "routes" contiene los archivos encargados de definir los servicios API correspondientes a cada módulo del sistema.

El archivo "server.js" constituye el archivo principal del backend, ya que permite iniciar el servidor, establecer la conexión con la base de datos e incorporar las diferentes rutas de los módulos.

El archivo "package.json" contiene la información del proyecto y las dependencias necesarias para su ejecución.

Esta organización facilita la separación de responsabilidades y permite realizar modificaciones o mantenimiento sobre los diferentes componentes sin afectar innecesariamente el resto de la aplicación.

La estructura modular también facilita la ejecución de pruebas sobre cada conjunto de rutas y permite mantener un código fuente organizado.

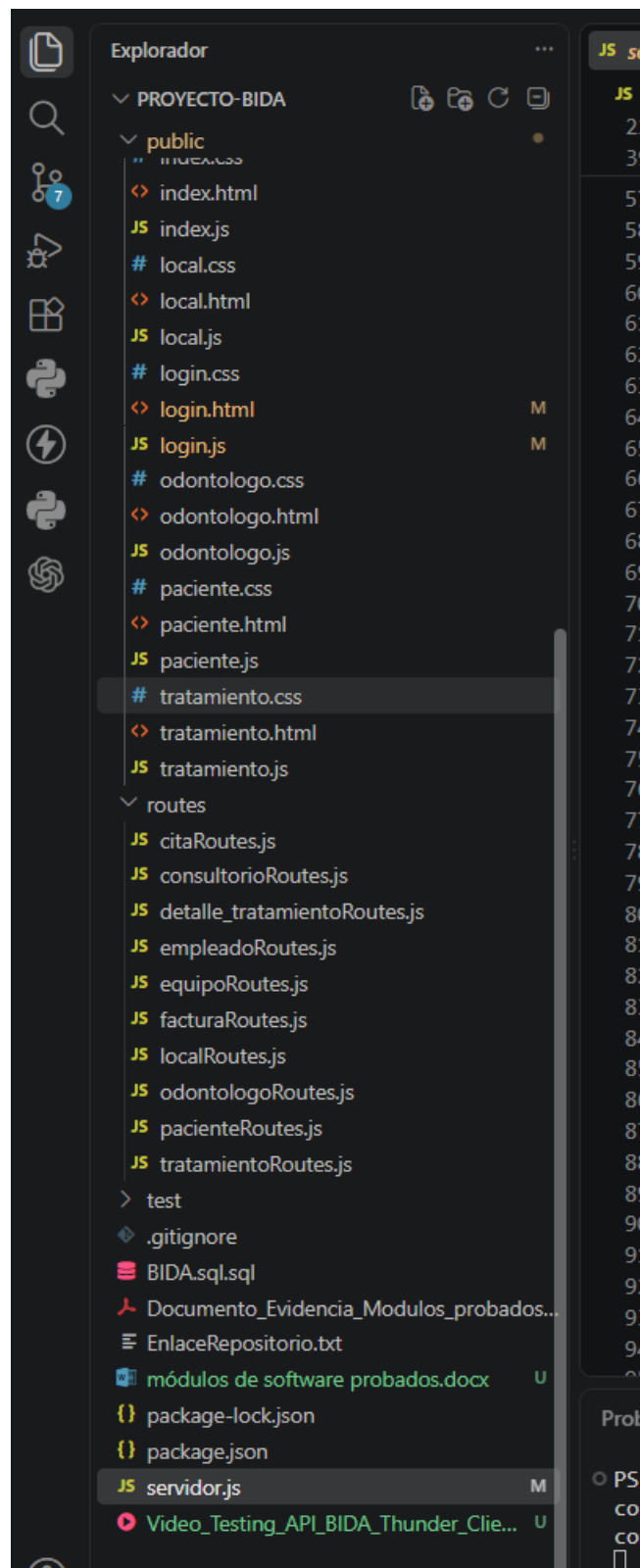


Figura 11. Organización de los archivos y carpetas principales del proyecto VIDA

15. BUENAS PRÁCTICAS DE PROGRAMACIÓN

Durante el desarrollo del proyecto BIDA – Odontología Especializada se aplicaron diferentes prácticas de organización y escritura de código con el propósito de facilitar su comprensión, mantenimiento y evolución.

El código fue organizado mediante archivos independientes para cada módulo, permitiendo separar las responsabilidades correspondientes a las diferentes funcionalidades del sistema.

Las rutas de la API fueron separadas en archivos individuales dentro de la carpeta "routes", utilizando nombres relacionados con cada módulo.

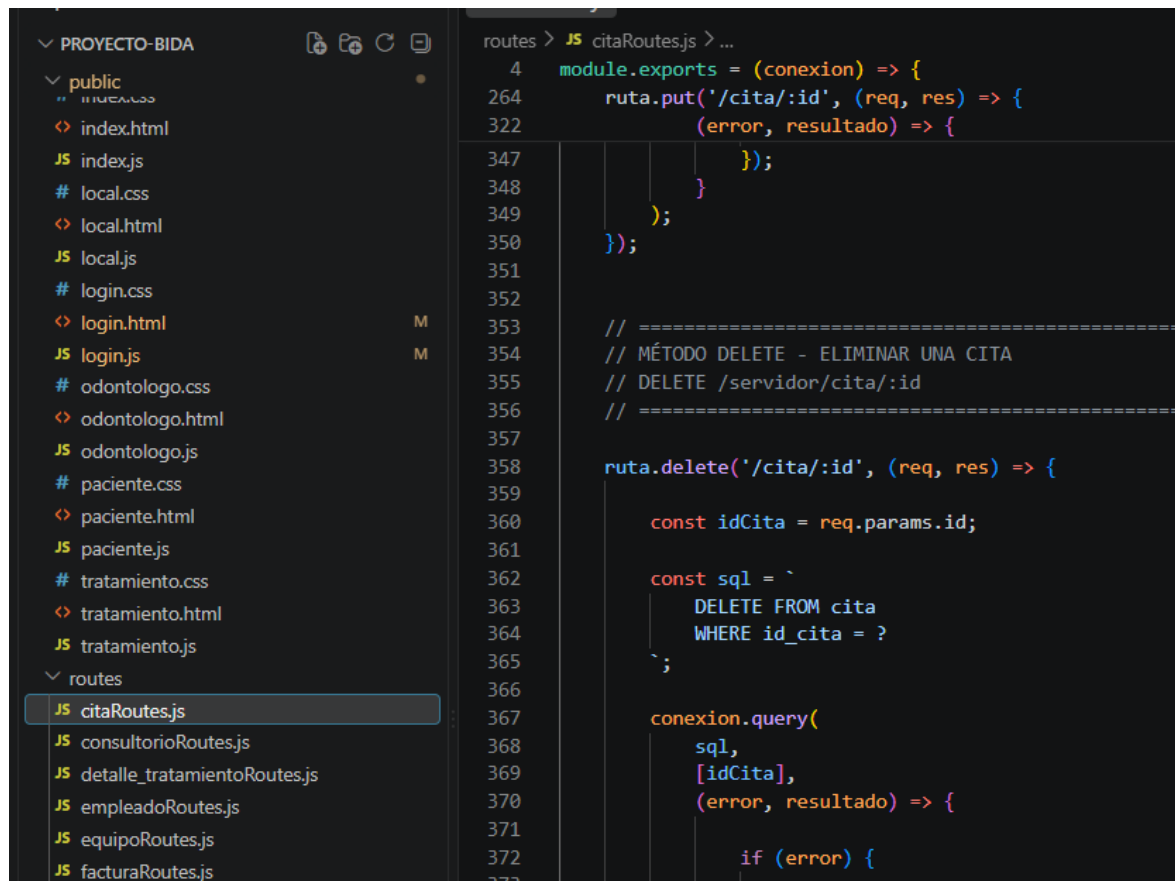
En las operaciones realizadas sobre la base de datos se utilizaron consultas parametrizadas para evitar la incorporación directa de valores provenientes de las solicitudes dentro de las consultas SQL.

También se implementaron validaciones de los datos recibidos en diferentes servicios, permitiendo controlar situaciones en las que faltan campos requeridos o los registros solicitados no existen.

Las respuestas de la API utilizan códigos de estado HTTP de acuerdo con el resultado de las operaciones. Entre ellos se utilizan códigos como 200 para operaciones exitosas, 201 para creación de registros, 400 para solicitudes con datos incorrectos, 404 cuando no se encuentra un recurso y 500 cuando ocurre un error interno.

La organización modular del proyecto permite realizar pruebas independientes y facilita la identificación de errores en cada componente.

Estas prácticas contribuyen a mantener un código estructurado y facilitan las actividades de mantenimiento y actualización del sistema.



```
routes > JS citaRoutes.js > ...
4  module.exports = (conexion) => {
264      ruta.put('/cita/:id', (req, res) => {
322          (error, resultado) => {
347              });
348          }
349      });
350  });
351
352
353  // =====
354  // MÉTODO DELETE - ELIMINAR UNA CITA
355  // DELETE /servidor/cita/:id
356  // =====
357
358  ruta.delete('/cita/:id', (req, res) => {
359
360      const idCita = req.params.id;
361
362      const sql = `
363          DELETE FROM cita
364          WHERE id_cita = ?
365      `;
366
367      conexion.query(
368          sql,
369          [idCita],
370          (error, resultado) => {
371
372              if (error) {
373                  // ...

```

Figura 12. Uso de consultas parametrizadas en las rutas de la API.

16. CONTROL DE VERSIONES

Para el control de versiones del proyecto BIDA – Odontología Especializada se utilizó Git como sistema de control de versiones y GitHub como plataforma para almacenar y administrar el repositorio remoto.

El repositorio permite conservar el código fuente del proyecto y realizar un seguimiento de los cambios efectuados durante el desarrollo.

El uso de un repositorio también facilita la disponibilidad del proyecto para su revisión y permite mantener una copia remota del código desarrollado.

El proyecto contiene los archivos correspondientes al frontend, backend, rutas de la API, configuración del servidor y demás componentes necesarios para su ejecución.

El repositorio utilizado corresponde al proyecto BIDA – Odontología Especializada y constituye una de las evidencias del proceso de desarrollo e integración de los módulos.

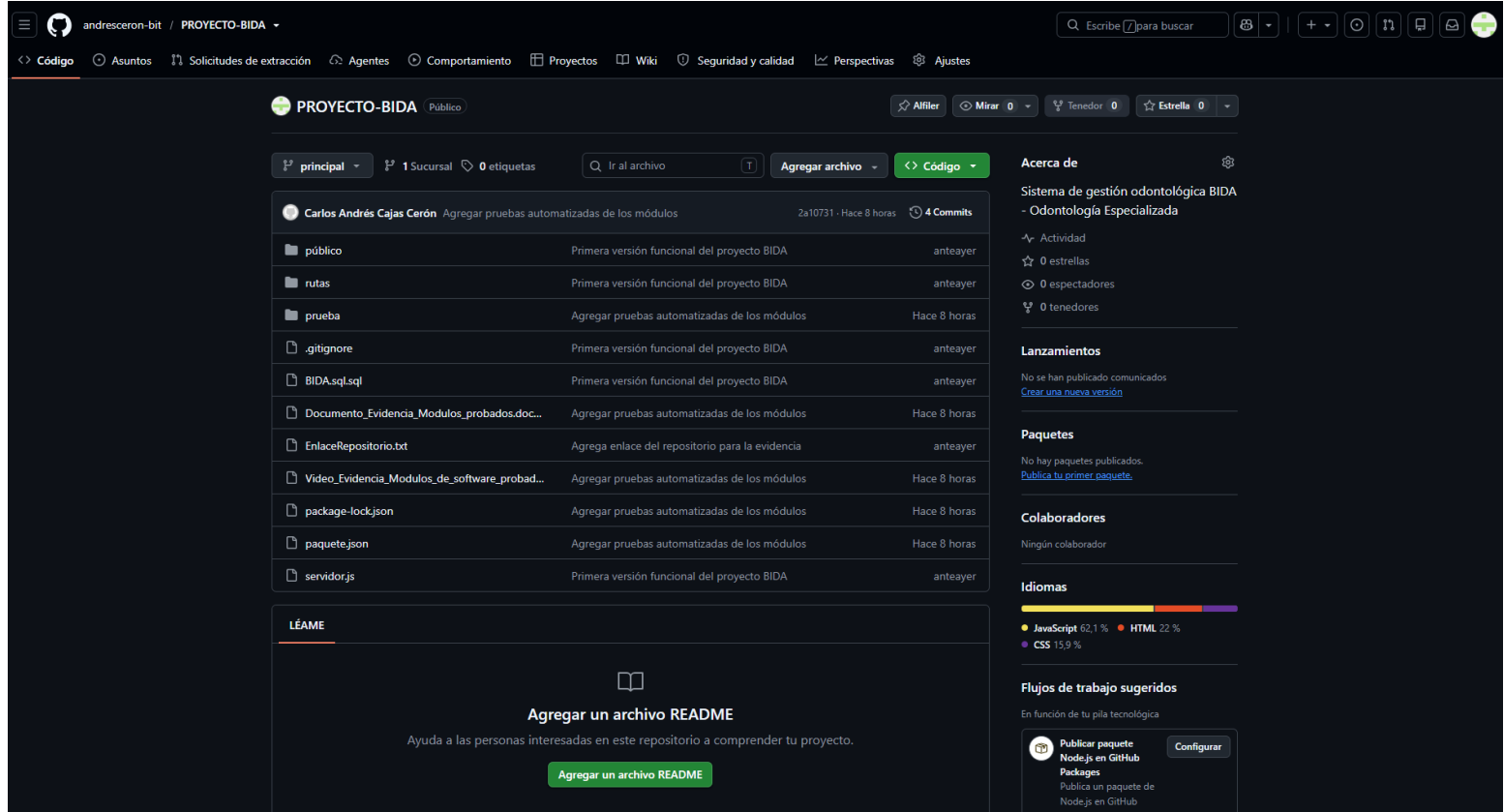


Figura 13. Repositorio GitHub utilizado para el control de versiones del proyecto BIDA.

17. PRUEBAS UNITARIAS

Como parte del desarrollo e integración de los módulos del proyecto BIDA – Odontología Especializada se realizaron pruebas automatizadas utilizando Jest.

Las pruebas permitieron verificar el funcionamiento de las diferentes rutas desarrolladas para los módulos de la aplicación.

Se evaluaron operaciones correspondientes a consultas, creación, actualización y eliminación de información, además de diferentes escenarios de error.

Las pruebas fueron organizadas en archivos independientes de acuerdo con los módulos correspondientes, permitiendo comprobar el comportamiento de las rutas de manera individual.

Entre los módulos evaluados se encuentran pacientes, odontólogos, empleados, facturación, tratamientos, locales, equipos, detalles de tratamientos, consultorios y citas.

La ejecución general de las pruebas permitió comprobar el funcionamiento de las rutas implementadas y detectar posibles errores antes de la integración final del sistema.

Como resultado de la ejecución general se obtuvieron:

10 suites de pruebas aprobadas.

132 pruebas aprobadas.

132 pruebas realizadas.

El resultado demuestra que las pruebas automatizadas ejecutadas sobre los módulos desarrollados finalizaron satisfactoriamente.

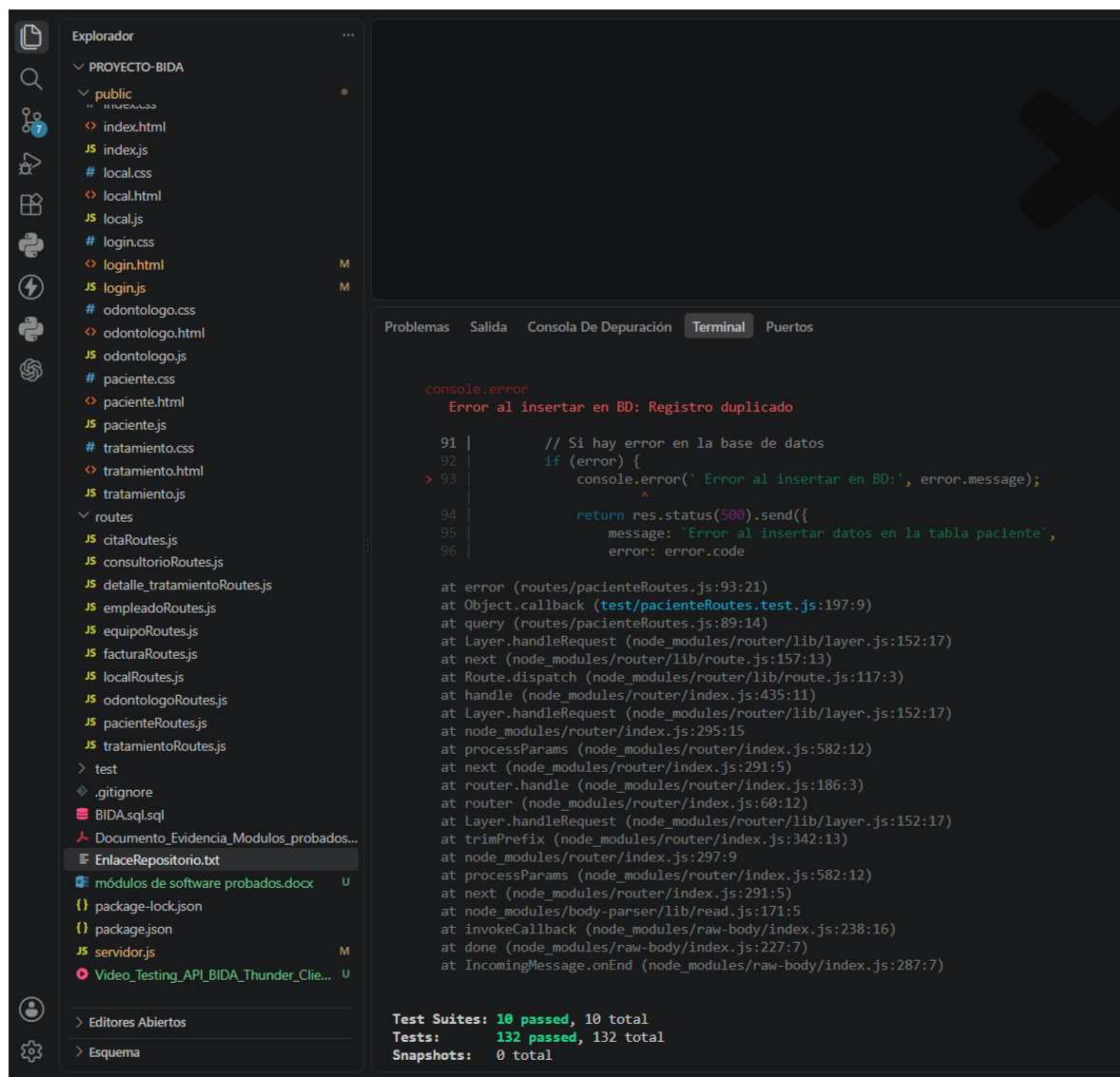


Figura 14. Resultado general de las pruebas automatizadas ejecutadas mediante Jest.

18. PRUEBAS DE FUNCIONAMIENTO

Además de las pruebas automatizadas realizadas mediante Jest, se realizaron pruebas de funcionamiento sobre la aplicación web con el propósito de comprobar la integración de los componentes principales.

Durante las pruebas se verificó el inicio del servidor local, la conexión con la base de datos, el acceso al sistema mediante el formulario de inicio de sesión y la navegación hacia el panel principal.

También se comprobó el acceso a los diferentes módulos disponibles desde la interfaz de la aplicación.

Las pruebas permitieron verificar que las solicitudes realizadas desde la interfaz fueran procesadas por los servicios correspondientes de la API y que las respuestas obtenidas fueran utilizadas para mostrar la información al usuario.

El funcionamiento integrado fue comprobado mediante el navegador y mediante herramientas de prueba utilizadas durante el desarrollo del proyecto.

Los resultados obtenidos permitieron comprobar que los componentes frontend, backend, API y base de datos se encuentran integrados dentro de una misma aplicación.

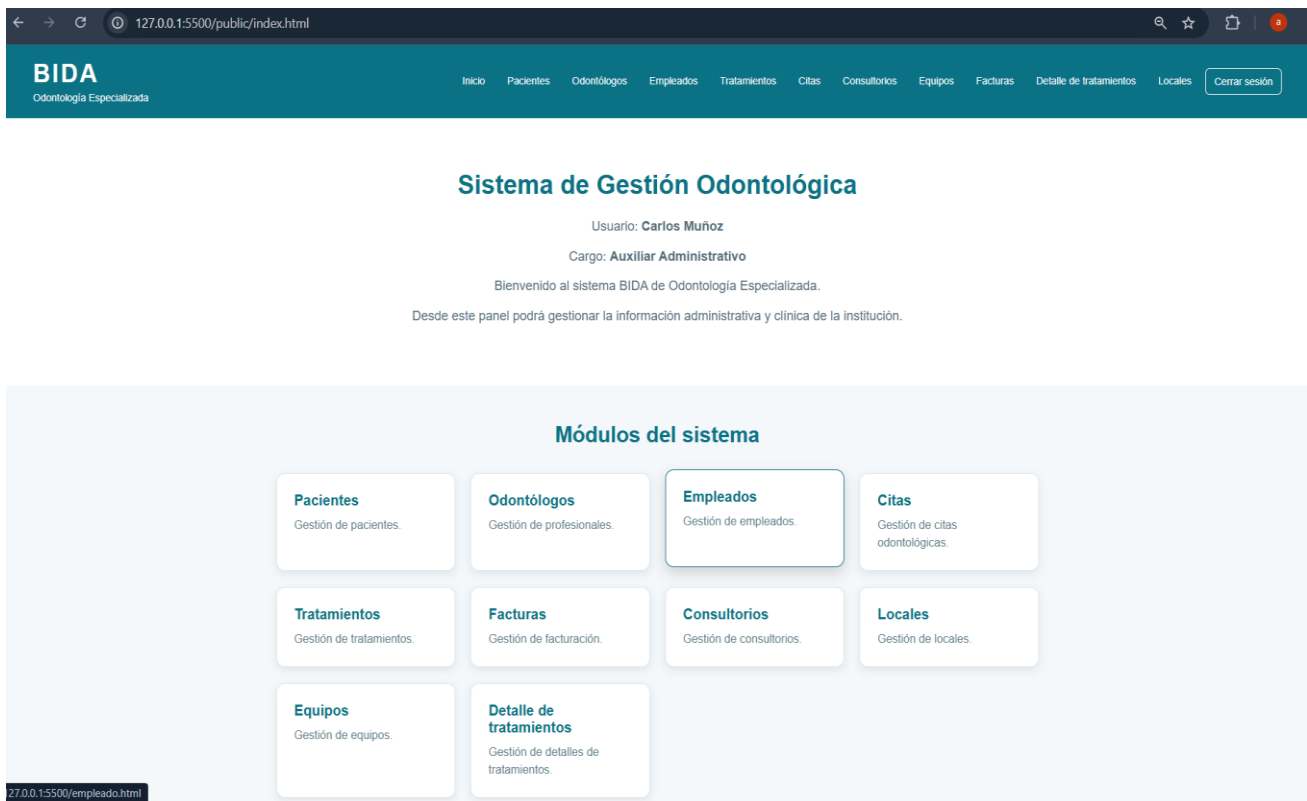


Figura 15. Aplicación BIDA funcionando e integrada desde el inicio de sesión hasta el panel principal.

19. EVIDENCIAS DE INTEGRACIÓN DE LOS MÓDULOS

La integración de los módulos del proyecto BIDA – Odontología Especializada se evidencia mediante la navegación desde el inicio de sesión hacia el panel principal y posteriormente hacia las diferentes funcionalidades de la aplicación.

Cada módulo se encuentra conectado con las rutas correspondientes de la API y con las tablas relacionadas de la base de datos.

Para comprobar esta integración se realizaron pruebas de navegación y consulta de información desde la interfaz web.

Los módulos disponibles permiten gestionar diferentes entidades relacionadas con la operación del sistema, manteniendo una estructura organizada y conectada mediante los servicios desarrollados.

La integración permite que el usuario utilice los diferentes componentes desde una única aplicación, mientras que el backend procesa las solicitudes y obtiene o modifica la información almacenada en MySQL.

BIDA

Odontología Especializada

Inicio

Pacientes

Odontólogos

Gestión de Pacientes

Administración de la información de los pacientes registrados en BIDA Odontología Especializada.

Registrar paciente

ID del paciente

Ingrese el ID

Documento de identidad

Número de documento

Nombre

Nombre del paciente

Apellido

Apellido del paciente

Fecha de nacimiento

dd/mm/aaaa

Género

Seleccione

Dirección

Dirección

Teléfono

Número de teléfono

Correo electrónico

correo@ejemplo.com

Guardar paciente

Limpiar

Consultar paciente

ID del paciente

Ingrese el ID

Consultar

Gestión de Tratamientos

Administración de los tratamientos odontológicos

Listado de Tratamientos

ID	Nombre del tratamiento	Descripción	Costo base
1	Limpieza Dental	Eliminación de placa bacteriana y cálculo dental.	\$ 80.000
2	Resina Dental	Restauración estética con resina fotocurada.	\$ 150.000
3	Extracción Dental	Extracción de una pieza dental.	\$ 120.000
4	Endodoncia	Tratamiento de conducto para conservar la pieza dental.	\$ 450.000
5	Ortodoncia	Corrección de la posición de los dientes mediante brackets.	\$ 2.500.000
6	Blanqueamiento Dental	Procedimiento estético para aclarar el color de los dientes.	\$ 350.000
7	Implante Dental	Reemplazo de un diente perdido mediante implante.	\$ 3.500.000
8	Prótesis Dental	Rehabilitación de dientes perdidos con prótesis.	\$ 1.800.000

Consultar Tratamiento por ID

ID del tratamiento

Ingrese el ID

Consultar tratamiento

Registrar Tratamiento

Nombre del tratamiento

Figura 16. Funcionamiento integrado de los módulos de BIDA desde la interfaz principal

20. MAPA DE NAVEGACIÓN

El mapa de navegación del proyecto BIDA – Odontología Especializada representa el recorrido que puede realizar el usuario dentro de la aplicación.

El proceso inicia en la página de inicio de sesión, donde el usuario debe ingresar el documento de identidad y el correo electrónico correspondiente a un empleado registrado en el sistema.

Una vez validada la información, el sistema dirige al usuario hacia el panel principal de la aplicación ("index.html").

Desde el panel principal se puede acceder a los diferentes módulos que conforman el sistema.

La navegación principal se puede representar de la siguiente manera:

Inicio de sesión → Validación del empleado → Panel principal → Módulos del sistema

Desde el panel principal se encuentran disponibles los módulos:

- Pacientes.
- Odontólogos.
- Empleados.
- Citas.

- Tratamientos.
- Detalle de tratamientos.
- Facturación.
- Consultorios.
- Equipos.
- Locales.

Cada módulo permite acceder a las funcionalidades correspondientes y comunicarse con los servicios de la API para gestionar la información almacenada en la base de datos.

La estructura de navegación permite que los diferentes componentes se encuentren disponibles desde una misma plataforma y facilita el acceso organizado a las funcionalidades del sistema.



Figura 17. Panel principal y estructura de navegación del sistema BIDA.

21. SEGURIDAD DE LA APLICACIÓN

El proyecto BIDA – Odontología Especializada incorpora mecanismos básicos de control de acceso y validación de información para proteger el funcionamiento de la aplicación.

El acceso al sistema se realiza mediante un formulario de inicio de sesión que solicita el documento de identidad y el correo electrónico del empleado.

El sistema consulta el empleado mediante un servicio de la API y posteriormente verifica que el correo electrónico ingresado corresponda al empleado encontrado.

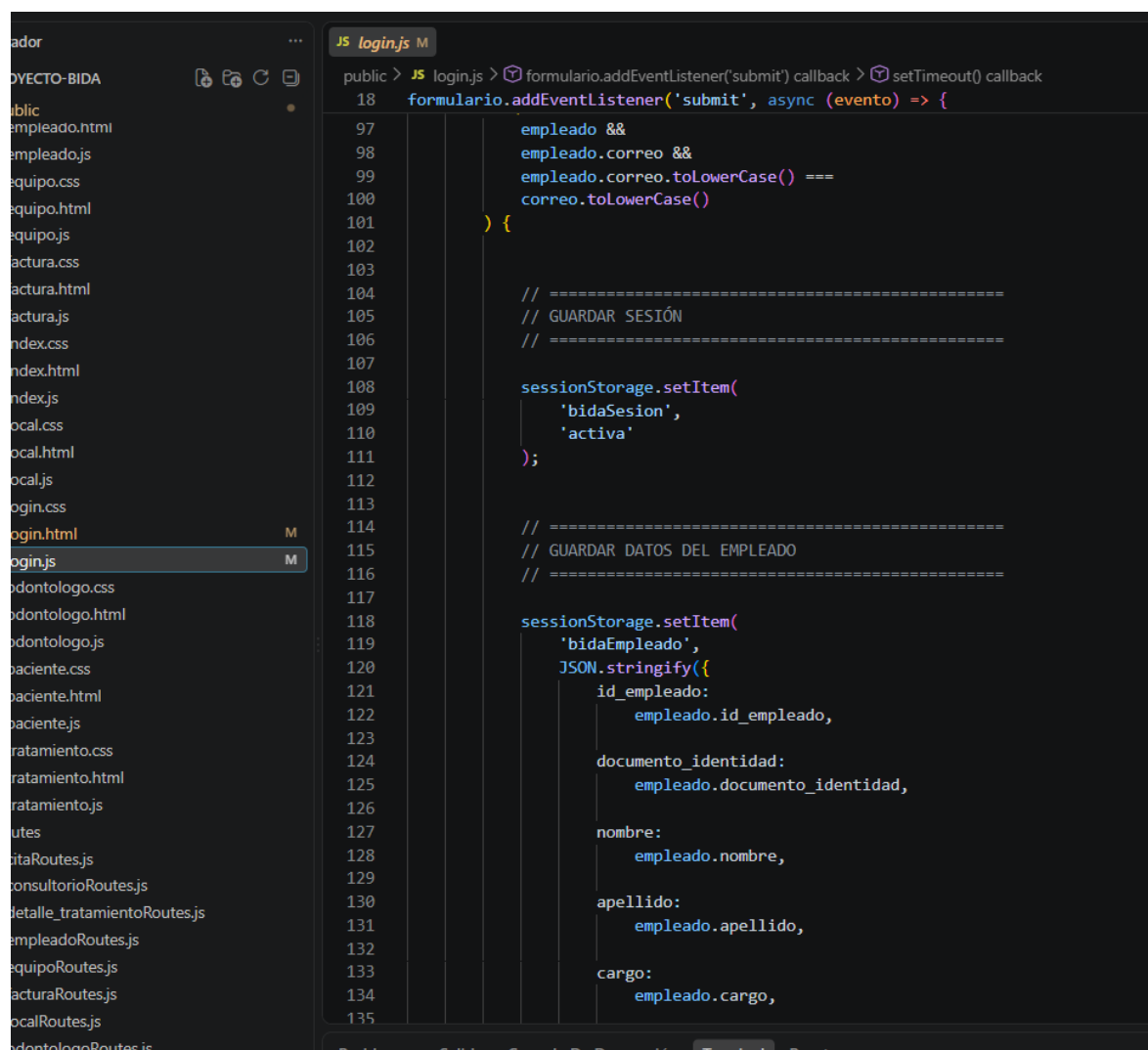
Cuando la información es válida, se establece una sesión utilizando "sessionStorage" y el usuario es dirigido al panel principal.

También se implementan validaciones en los servicios de la API para controlar los datos recibidos y evitar operaciones con información incompleta.

Las consultas a la base de datos utilizan parámetros en las operaciones que reciben información desde las solicitudes, evitando incorporar directamente los valores proporcionados por el usuario dentro de las consultas SQL.

Adicionalmente, las rutas manejan diferentes códigos de respuesta HTTP para identificar situaciones exitosas, registros inexistentes, datos incompletos y errores internos.

Estos mecanismos contribuyen a controlar el acceso y a mantener una comunicación organizada entre la interfaz, la API y la base de datos.



```
public > JS login.js > formulario.addEventListener('submit') callback > setTimeout() callback
18 formulario.addEventListener('submit', async (evento) => {
97     empleado &&
98     empleado.correo &&
99     empleado.correo.toLowerCase() ===
100     correo.toLowerCase()
101 } {
102
103
104 // =====
105 // GUARDAR SESIÓN
106 // =====
107
108 sessionStorage.setItem(
109     'bidaSesion',
110     'activa'
111 );
112
113
114 // =====
115 // GUARDAR DATOS DEL EMPLEADO
116 // =====
117
118 sessionStorage.setItem(
119     'bidaEmpleado',
120     JSON.stringify({
121         id_empleado:
122             empleado.id_empleado,
123
124         documento_identidad:
125             empleado.documento_identidad,
126
127         nombre:
128             empleado.nombre,
129
130         apellido:
131             empleado.apellido,
132
133         cargo:
134             empleado.cargo,
135     })
136 }
```

Figura 18. Validación de acceso y establecimiento de sesión en el sistema BIDA.

22. AMBIENTES DE DESARROLLO Y PRUEBAS

El proyecto BIDA – Odontología Especializada fue desarrollado y probado en un ambiente local.

El ambiente de desarrollo utiliza Visual Studio Code como entorno de programación, Node.js como entorno de ejecución del servidor, Express para la implementación de los servicios web y MySQL como sistema gestor de base de datos.

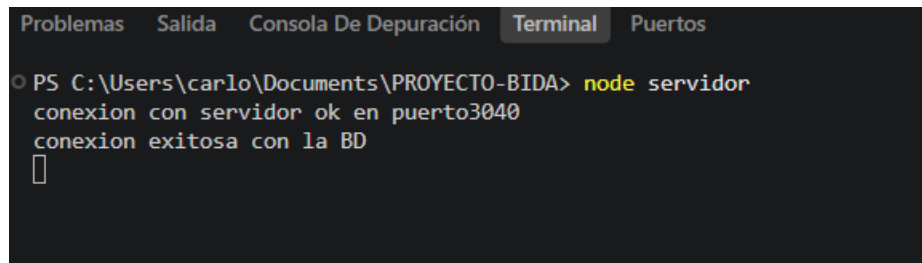
El servidor de la aplicación se encuentra configurado en el puerto 3040, mientras que la conexión utilizada para la base de datos MySQL se encuentra configurada en el puerto 3309.

Para las pruebas automatizadas se utilizó Jest, permitiendo comprobar el funcionamiento de las rutas correspondientes a los diferentes módulos.

También se utilizó Thunder Client para realizar pruebas de los servicios mediante solicitudes HTTP y verificar las respuestas generadas por la API.

Las pruebas de funcionamiento de la aplicación se realizaron mediante el navegador utilizando el servidor Node.js/Express, verificando el inicio de sesión, navegación entre módulos, consulta de información y comunicación con los servicios de la API.

El ambiente de desarrollo permitió realizar modificaciones, ejecutar pruebas y verificar el funcionamiento de los componentes antes de la entrega del proyecto.

A screenshot of a Visual Studio Code terminal window. The terminal has tabs at the top: 'Problemas', 'Salida', 'Consola De Depuración', 'Terminal' (which is active), and 'Puertos'. The terminal text shows a PowerShell prompt 'PS C:\Users\carlo\Documents\PROYECTO-BIDA>' followed by the command 'node servidor'. The output of the command is 'conexion con servidor ok en puerto3040' and 'conexion exitosa con la BD'. A cursor is visible on the line following the output.

```
Problemas  Salida  Consola De Depuración  Terminal  Puertos
PS C:\Users\carlo\Documents\PROYECTO-BIDA> node servidor
conexion con servidor ok en puerto3040
conexion exitosa con la BD
█
```

23. REPOSITORIO DEL PROYECTO

El código fuente del proyecto BIDA – Odontología Especializada se encuentra almacenado en un repositorio remoto utilizando GitHub.

El repositorio contiene los archivos necesarios para identificar la estructura y componentes principales del proyecto, incluyendo el código correspondiente al frontend, backend, rutas de los servicios y configuración del servidor.

El uso del repositorio permite mantener una copia del proyecto, facilitar su revisión y conservar el historial de cambios realizados durante el proceso de desarrollo.

La dirección del repositorio es:

[PEGAR AQUÍ EL ENLACE REAL DEL REPOSITORIO DE BIDA]

El repositorio constituye también una evidencia del uso de herramientas de control de versiones durante el desarrollo del proyecto.

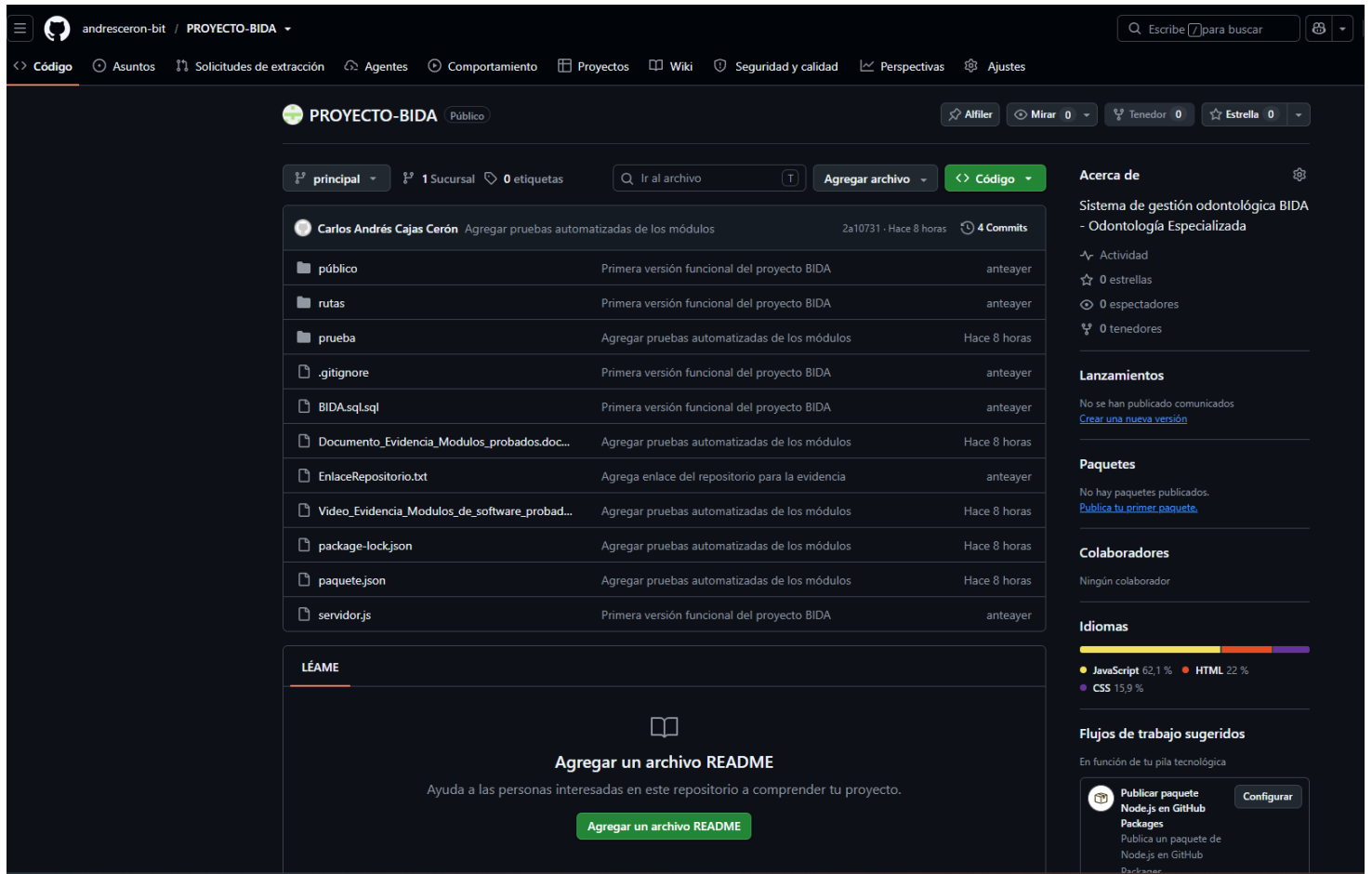


Figura 19. Repositorio remoto del proyecto BIDA en GitHub.

24. CONCLUSIONES

El desarrollo del proyecto BIDA – Odontología Especializada permitió integrar diferentes módulos funcionales dentro de una aplicación web orientada a la gestión de información odontológica.

La integración entre frontend, backend, servicios API y base de datos permitió construir una aplicación en la que las operaciones realizadas desde la interfaz son procesadas mediante servicios desarrollados con Node.js y Express y almacenadas en MySQL.

La organización de las rutas por módulos facilitó el desarrollo, mantenimiento y prueba de cada componente de la aplicación.

El sistema de inicio de sesión permitió integrar el control de acceso con el panel principal, desde donde el usuario puede acceder a las diferentes funcionalidades del sistema.

Las pruebas automatizadas realizadas mediante Jest permitieron verificar el comportamiento de los módulos y sus diferentes operaciones. La ejecución general obtuvo 10 suites aprobadas y 132 pruebas aprobadas.

Las pruebas realizadas durante el desarrollo permitieron comprobar la comunicación entre los componentes y detectar posibles errores antes de la entrega del proyecto.

El uso de Git y GitHub permitió mantener el código fuente organizado mediante un sistema de control de versiones y disponer de un repositorio remoto para su consulta y revisión.

Finalmente, el proyecto permitió aplicar los conocimientos relacionados con desarrollo web, servicios API, bases de datos, integración de módulos, pruebas y control de versiones, obteniendo como resultado una aplicación funcional y estructurada de acuerdo con los propósitos establecidos para el sistema BIDA – Odontología Especializada.